

TUGAS 2

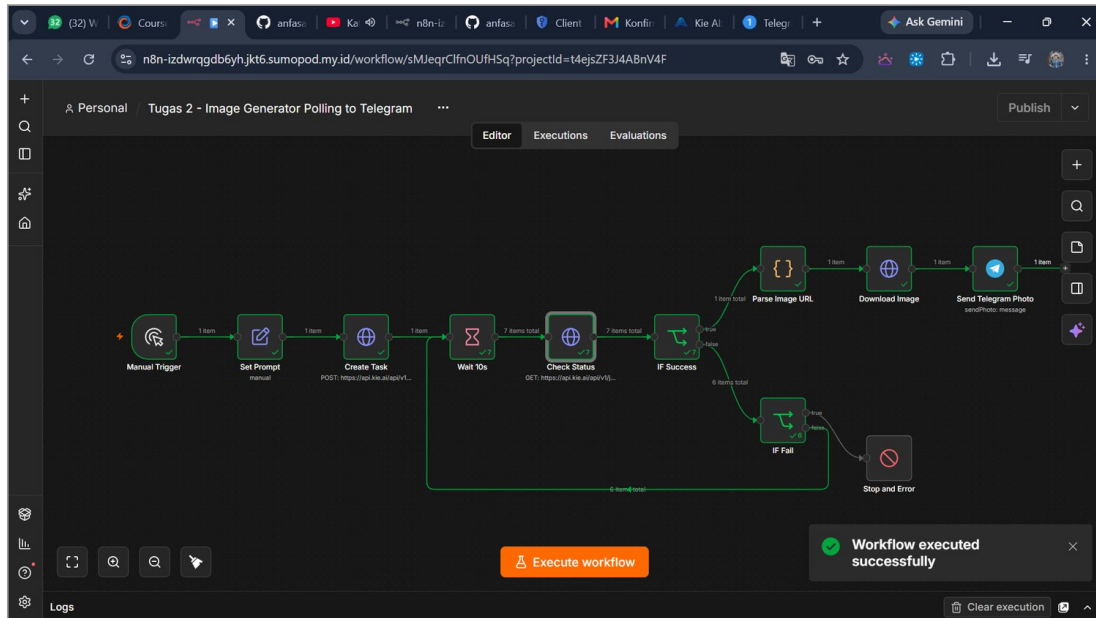
Otomasi Image Generator dengan Polling Loop di n8n

HTTP Request, Wait, IF, Code Node, dan Telegram

Nama	Anfasa Umar
NIM	0110224130
Tanggal	9 Oktober 2026
Repositori GitHub	https://github.com/anfasaumar/n8n-Arkansas
File workflow	Tugas 2 - Image Generator Polling to Telegram.json

1. Gambaran Umum Workflow

Workflow ini membuat gambar dari teks prompt memakai API Kie.ai, lalu mengirim hasilnya ke Telegram. Pembuatan gambar di Kie.ai berjalan asinkron: permintaan pertama hanya mengembalikan **taskId**, sedangkan gambarnya baru siap beberapa detik kemudian. Karena itu workflow memakai **polling loop**, yaitu menunggu 10 detik, mengecek status task, dan mengulang sampai statusnya **success**.



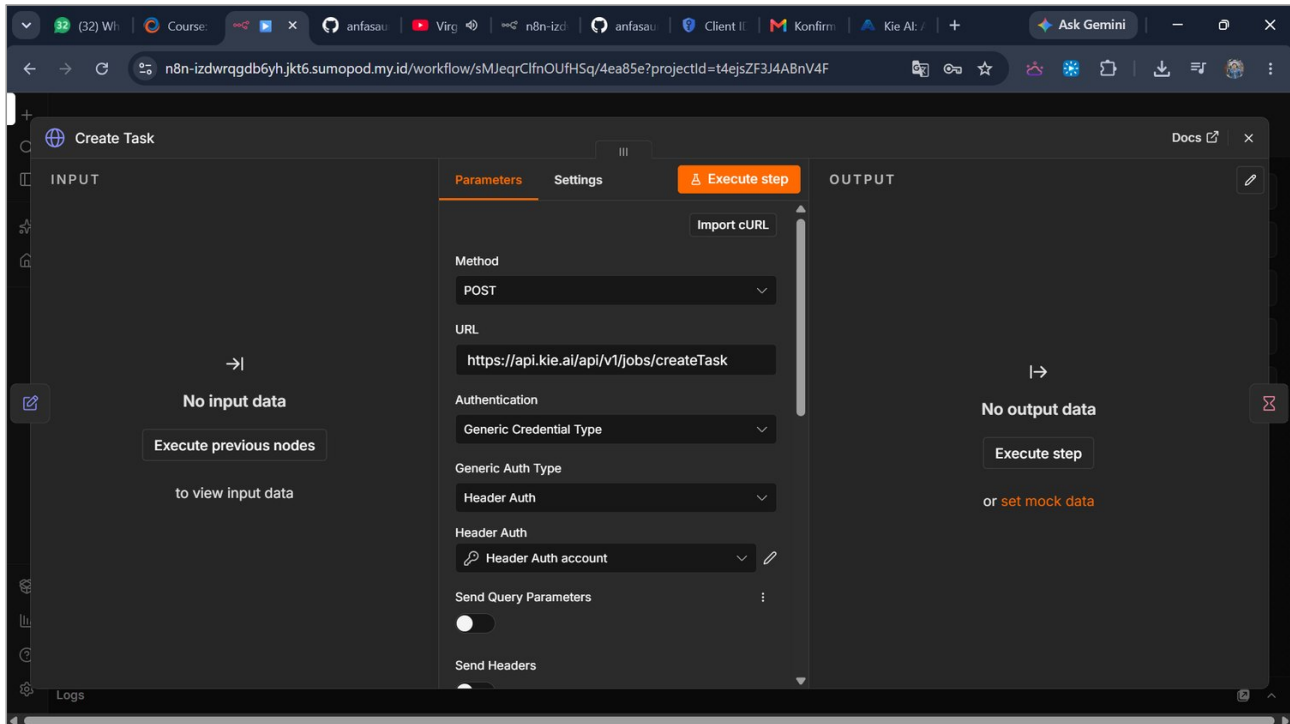
Gambar 1. Workflow lengkap setelah dijalankan. Angka di antara node menunjukkan jumlah item yang lewat; Wait, Check Status, dan IF Success berjalan 7 kali, IF Fail 6 kali (6 kali status belum success lalu kembali ke Wait, putaran ke-7 lanjut ke Parse Image URL).

Urutan node:

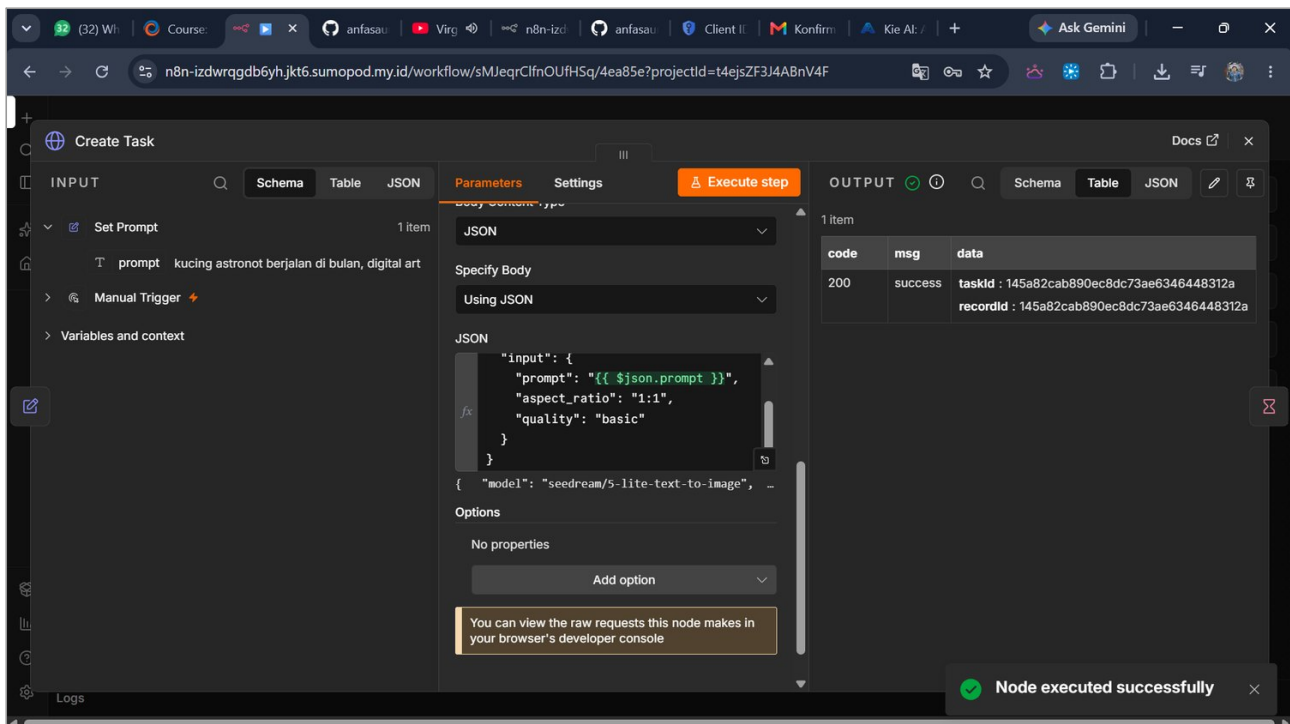
No	Node	Tipe	Fungsi
1	Manual Trigger	Trigger	Memulai workflow secara manual.
2	Set Prompt	Edit Fields (Set)	Menyimpan teks prompt gambar di field prompt .
3	Create Task	HTTP Request (POST)	Mengirim prompt ke Kie.ai dan menerima taskId .
4	Wait 10s	Wait	Jeda 10 detik sebelum setiap pengecekan status.
5	Check Status	HTTP Request (GET)	Mengecek status task berdasarkan taskId .
6	IF Success	IF	Lanjut hanya jika state sama dengan success ; jika tidak, ke IF Fail.
7	IF Fail / Stop and Error	IF / Stop and Error	Jika state fail workflow berhenti dengan pesan error; jika belum, kembali ke Wait (loop).
8	Parse Image URL	Code (JavaScript)	Mem-parsing resultJson dan mengambil URL gambar yang valid.
9	Download Image	HTTP Request (GET)	Mengunduh gambar sebagai file biner.
10	Send Telegram Photo	Telegram	Mengirim file gambar ke chat pengguna.

2. Node HTTP Request (POST) - Create Task

Node ini mengirim prompt ke endpoint **`https://api.kie.ai/api/v1/jobs/createTask`** dengan metode POST. Autentikasi memakai *Generic Credential Type* jenis *Header Auth*: nama header **Authorization** dan nilai **Bearer** diikuti API key. Body dikirim sebagai JSON berisi nama model dan **input** (prompt, aspect_ratio, quality). Nilai prompt diambil dari node Set Prompt lewat ekspresi **`{{ $json.prompt }}`**.



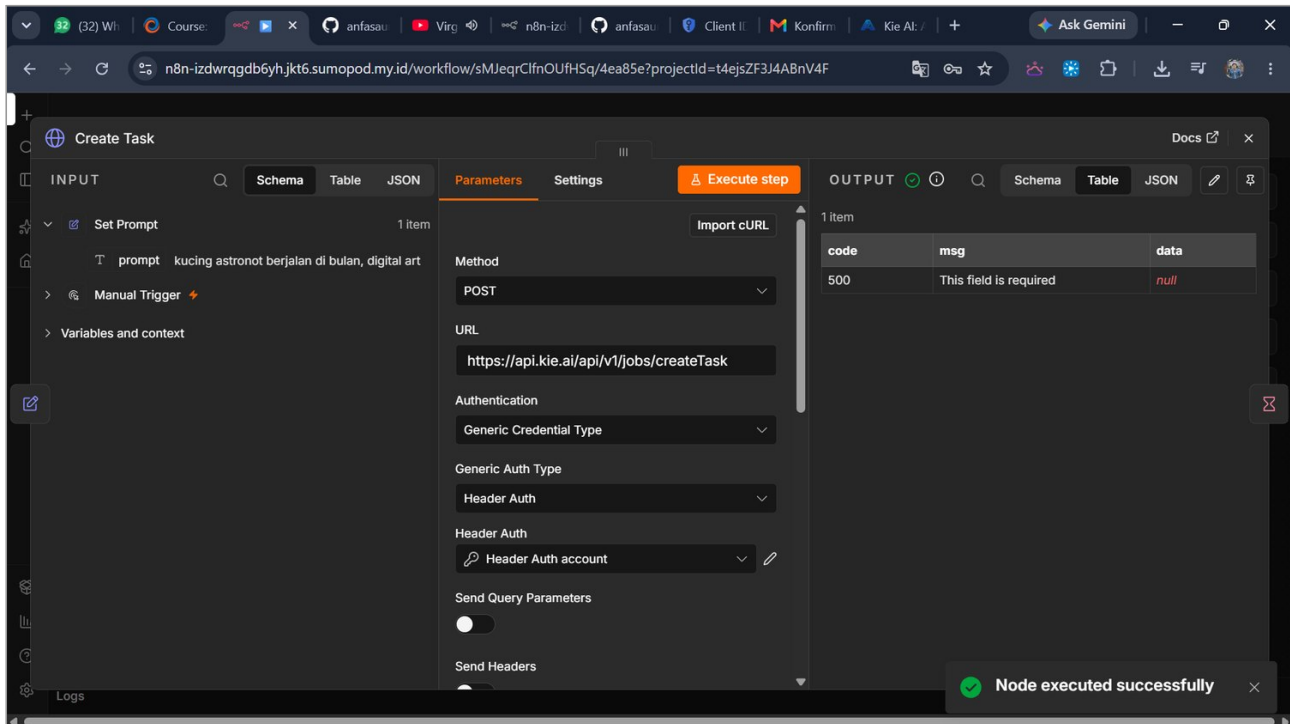
Gambar 2. Konfigurasi Create Task: method POST, URL createTask, dan autentikasi Header Auth.



Gambar 3. Body JSON dan hasil eksekusi: kode 200 dengan taskId yang dikembalikan API.

Catatan debugging

Pada percobaan pertama, API membalas kode 500 dengan pesan *This field is required*. Penyebabnya, body hanya berisi prompt, padahal model seedream/5-lite-text-to-image mewajibkan field **aspect_ratio** dan **quality**. Setelah kedua field ditambahkan, API membalas kode 200 dan memberikan taskId. Perlu diingat bahwa node HTTP Request tetap ditandai berhasil (centang hijau) walaupun isi balasan API berisi error, sehingga isi Output harus selalu diperiksa.



The screenshot shows the n8n workflow editor interface. On the left, the 'INPUT' section contains a 'Set Prompt' node with the prompt 'kucing astronot berjalan di bulan, digital art'. The main workspace shows an 'HTTP Request' node configured with the following parameters:

- Method: POST
- URL: <https://api.kie.ai/api/v1/jobs/createTask>
- Authentication: Generic Credential Type
- Generic Auth Type: Header Auth
- Header Auth: Header Auth account
- Send Query Parameters: Off
- Send Headers: Off

The 'OUTPUT' section on the right shows the result of the execution:

code	msg	data
500	This field is required	null

A green checkmark and the message 'Node executed successfully' are visible at the bottom right of the workflow editor.

Gambar 4. Balasan error 500 sebelum field `aspect_ratio` dan `quality` ditambahkan.

3. Polling Loop: Node Wait dan HTTP Request (GET)

Setelah taskId diterima, workflow tidak langsung mengambil gambar karena task belum tentu selesai. Node **Wait 10s** diatur untuk menunda 10 detik, kemudian node **Check Status** melakukan GET ke **https://api.kie.ai/api/v1/jobs/recordInfo** dengan query parameter **taskId** yang nilainya diambil dari node Create Task lewat ekspresi `{{ $('Create Task').item.json.data.taskId }}`. Credential Header Auth yang sama dipakai kembali.

Loop terbentuk dari sambungan balik: jika status belum success dan belum fail, alur dari IF Fail (cabang false) kembali ke Wait 10s, lalu mengecek status lagi. Pada percobaan ini loop berjalan 7 kali (lihat Gambar 1).

The screenshot displays the configuration for the 'Check Status' node in a workflow editor. The node is configured with a 'Wait 10s' node, a 'Create Task' node, and an 'IF Fail' node. The 'Check Status' node is configured with a 'Header Auth' section, a 'Send Query Parameters' section, and a 'Specify Query Parameters' section. The 'Specify Query Parameters' section shows a query parameter named 'taskId' with a value expression: `{{ $('Create Task').item.json.data.taskId }}`. The 'OUTPUT' section shows the result of the GET request, including a 'code' of 200, a 'msg' of 'success', and a 'data' object containing 'taskId', 'model', 'state', 'param', 'resultJson', 'failCode', 'failMsg', 'costTime', 'completeTime', 'createTime', 'creditsConsumed', and 'operationType'. A green checkmark indicates the node executed successfully.

Gambar 5. Konfigurasi Check Status dan Output: state = success, serta resultJson berisi resultUrls.

4. Node IF - Memastikan Status success

Node **IF Success** memeriksa apakah `{{ $json.data.state }}` is equal to **success**. Jika benar, data keluar lewat cabang **true** menuju Parse Image URL (tahap pengunduhan). Jika tidak, data keluar lewat cabang **false** menuju IF Fail. Dengan cara ini workflow tidak pernah mengunduh gambar sebelum API menyatakan task selesai.

Node **IF Fail** melengkapi logika ini: jika state bernilai **fail**, workflow dihentikan oleh node Stop and Error dengan pesan jelas, sehingga loop tidak berjalan tanpa akhir ketika pembuatan gambar gagal.

The screenshot shows the n8n interface for an 'IF Success' node. The 'Conditions' tab is active, showing a condition: `{{ $json.data.state }}` is equal to `success`. The 'Execute step' button is visible. The 'OUTPUT' panel on the right shows the results of the execution for the 'True Branch (1 item)'.

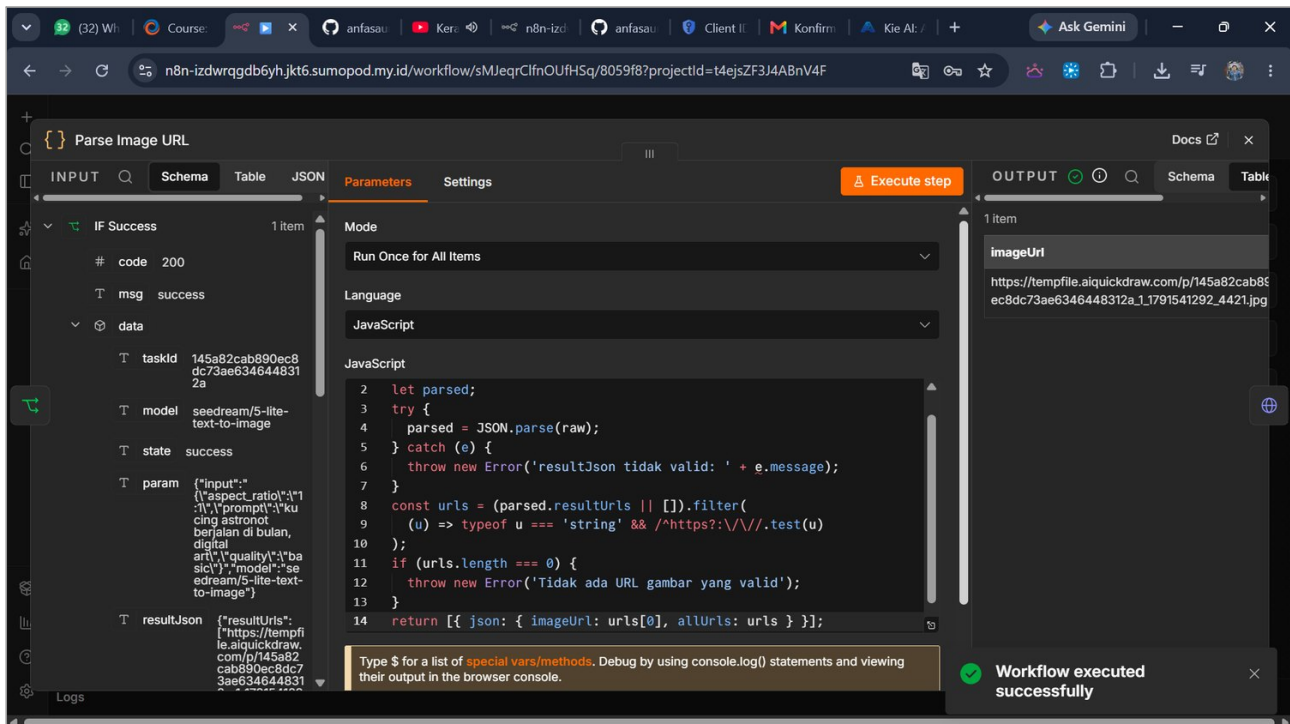
code	msg	data
200	success	<pre>{ "taskid": "145a82cab890ec8dc73ae6346448312a", "model": "seedream/5-lite-text-to-image", "state": "success", "param": { "input": { "prompt": "berjalan di bulan, digital art, quality", "model": "seedream/5-lite-text-to-image" } }, "resultUrl": "https://tempfile.aiquickdraw.com/p/145a82cab890ec8dc73ae6346448312a" }</pre>

The 'False Branch' is empty. The 'Node executed successfully' status is shown at the bottom right.

Gambar 6. Kondisi IF Success dan hasil eksekusi pada True Branch (1 item).

5. Code Node (JavaScript) - Parse Image URL

Field **resultJson** dari API berisi teks berformat JSON, bukan objek. Isinya terlihat seperti objek, tetapi n8n menerimanya sebagai string, sehingga harus diubah dulu dengan **JSON.parse** sebelum **resultUrls** dapat dibaca sebagai array. Kode kemudian menyaring array tersebut agar hanya URL berupa teks yang diawali http atau https yang diambil. Jika JSON tidak valid atau tidak ada URL yang valid, kode melempar error sehingga masalahnya langsung terlihat. URL pertama dikirim ke node berikutnya sebagai **imageUrl**.



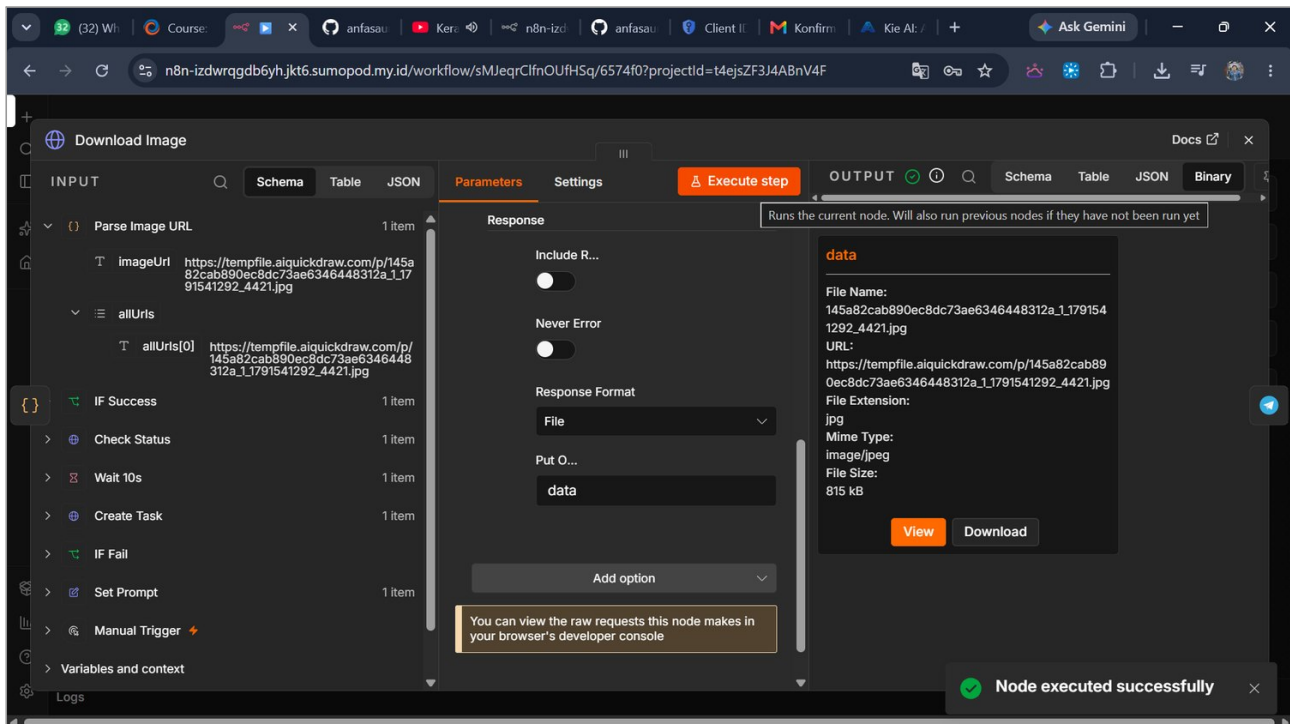
Gambar 7. Kode JavaScript pada node Parse Image URL dan output imageUrl.

Kode lengkap:

```
const raw = $input.first().json.data.resultJson;
let parsed;
try {
  parsed = JSON.parse(raw);
} catch (e) {
  throw new Error('resultJson tidak valid: ' + e.message);
}
const urls = (parsed.resultUrls || []).filter(
  (u) => typeof u === 'string' && /^https?:\/\//.test(u)
);
if (urls.length === 0) {
  throw new Error('Tidak ada URL gambar yang valid');
}
return [{ json: { imageUrl: urls[0], allUrls: urls } }];
```

6. Node HTTP Request - Download Image

Node ini melakukan GET ke URL gambar (`{{ $json.imageUrl }}`). Pada *Options > Response*, **Response Format** diatur ke **File** agar gambar diterima sebagai data biner (binary) dengan nama properti **data**. Tanpa pengaturan ini, n8n akan membaca gambar sebagai teks dan Telegram tidak dapat menerimanya. Pada Output terlihat file berformat JPG, Mime Type image/jpeg, berukuran 815 kB.



Gambar 8. Response Format = File dan hasil unduhan pada tab Binary.

7. Node Telegram - Send Photo

Node terakhir mengirim gambar ke pengguna. Pengaturannya: Resource **Message**, Operation **Send Photo**, **Chat ID** diisi ID Telegram penerima, **Binary File** diaktifkan, dan **Input Binary Field** diisi **data** sesuai nama properti dari node Download Image. Credential memakai token bot dari BotFather (disimpan sebagai credential n8n, tidak ditulis di dalam workflow). Output **ok: true** menandakan Telegram menerima dan mengirim pesan.

The screenshot shows the n8n web interface with the 'Send Telegram Photo' node configured. The 'INPUT' tab on the left shows a 'Download Image' node connected to the 'data' input field. The 'Parameters' tab in the center shows the following settings: Credential is 'Telegram account', Resource is 'Message', Operation is 'Send Photo', Chat ID is '7327346678', Binary File is checked, and Input Binary Field is 'data'. The 'OUTPUT' tab on the right shows the execution result in JSON format:

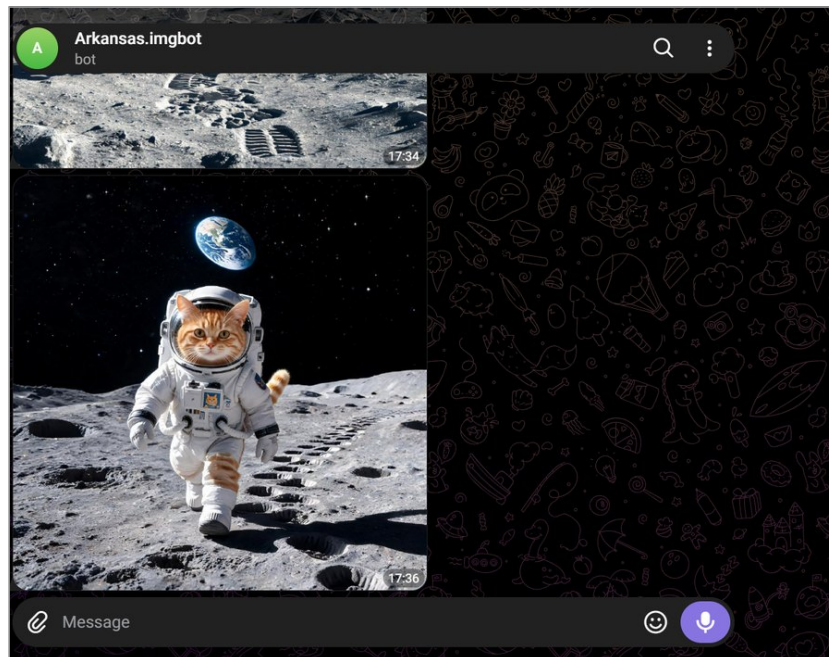
```
{  "ok": true,  "result": {    "message_id": 2,    "from": {      "id": 8981018602,      "is_bot": true,      "first_name": "Arkansas.imgbot",      "username": "Arkansas_imgbot"    },    "chat": {      "id": 7327346678,      "first_name": "Anfasa",      "last_name": "Umar",      "username": "Anfsuwr",      "type": "private"    }  }}}
```

A notification at the bottom right states 'Workflow executed successfully'.

Gambar 9. Konfigurasi Send Telegram Photo dan Output (ok: true, message_id).

8. Hasil Akhir

Setelah seluruh workflow dijalankan lewat Execute workflow, gambar hasil prompt "kucing astronot berjalan di bulan, digital art" diterima di Telegram dari bot Arkansas.imgbot.



Gambar 10. Gambar hasil diterima di Telegram.

9. Kesimpulan

Workflow berhasil mengotomatiskan proses dari prompt teks sampai gambar diterima di Telegram. Polling loop dengan Wait dan HTTP Request GET menangani proses asinkron API, node IF memastikan pengunduhan hanya terjadi saat status success (dengan penanganan status fail agar tidak terjadi loop tak berujung), Code Node mem-parsing resultJson untuk mengambil URL gambar yang valid, dan node Telegram mengirim file gambar hasil unduhan langsung ke pengguna.

Kode sumber workflow (file JSON) tersedia di repositori:
<https://github.com/anfasaumar/n8n-Arkansas>